

From Organizational Policy to Runtime Risk: Consequence-Aware Access Control for Model Context Protocol Servers

Tomer Ovadya

Ben-Gurion University of the Negev
Software and Information Systems Engineering
Beer-Sheva, Israel
ovadyat@post.bgu.ac.il

Asaf Shabtai

Ben-Gurion University of the Negev
Software and Information Systems Engineering
Beer-Sheva, Israel
shabtaia@bgu.ac.il

ABSTRACT

[To be written: abstract — write last, once the evaluation section is fixed.]

CCS CONCEPTS

• **Security and privacy** → **Distributed systems security**; *Access control*; • **Computing methodologies** → *Intelligent agents*.

KEYWORDS

Model Context Protocol; AI agents; risk scoring; security categorization; data classification policy; LLM security; access control

1 INTRODUCTION

AI agents no longer only generate text; they act. They read data, modify files, update databases, and call external services for their users. The Model Context Protocol (MCP) [3] standardizes this exchange: a server publishes tools, and a connected agent discovers and invokes them over JSON-RPC [35]. The public server ecosystem is growing quickly [21], and a study of 1,899 open-source servers found recurring security weaknesses [18]. An agent’s decision now lands directly on the systems and resources behind those tools. MCP does carry a risk vocabulary, but a coarse one: a tool descriptor may declare `readOnlyHint`, `destructiveHint`, `idempotentHint` and `openWorldHint` [35]. These are booleans, declared per tool rather than per call, and asserted by the server about itself; the specification tells clients to treat them as untrusted unless the server is trusted. Its remaining answer to risk is a human who can deny the invocation. A hint labels a capability. It says nothing about which resource a given call reaches [8, 21].

Most MCP and agent security work studies the agent’s side: how a malicious or compromised server subverts the agent connecting to it [26, 45]. A smaller line of work defends the server. Before deployment, static analysis maps implementation weaknesses to attack patterns and scores a repository [29], or audits declared capabilities for over-privilege [23]. At runtime, access control has been expressed as a capability manifest enforced by a sandbox [8], and hybrid analysis pairs a call’s parameters with its observed execution [19]. Where these methods score, the unit is a server or a capability, never an invocation. They also defend the opposite direction: a capability sandbox confines a faulty or malicious server to protect the host [8], while the assets at stake here are the ones the server itself owns.

We study *misuse*: the inappropriate use of a legitimate capability. Misuse needs no malicious agent and no malicious server. It starts

with an operator who states a task loosely, delegates too broadly, or grants more permission than the task needs; measurements of MCP servers report broad system access and over-privileged tools [23, 32]. It starts with the model, which can pick an available but wrong tool [53], or know a risk in the abstract and still miss it in a live trace [49]. It needs no single harmful call: legitimate tools chain into a workflow that is harmful in combination, driven by instructions hidden in data the agent reads during an ordinary task [56]. Prompt injection [54] and agentic misalignment [33] reach the same place by other routes, but are not our subject. We do not ask why the agent issued the call. We ask what the call would do. The same `read_file` is routine against a build log and a disclosure against a credentials store. Our unit of analysis is the whole invocation: the tool, the resource it reaches, the arguments, the execution context, and the agent’s recent activity.

Access control decides whether a principal may reach a resource. Role- and attribute-based models [22, 42] decide whether an operation is permitted; they attach no magnitude to a permitted one. MCP widens the gap: servers hold authorization as persistent state instead of re-checking each call [25], and agent frameworks expose a capability without re-authorizing the concrete call it emits [34]. Authentication says who is calling, authorization says what they may do. Neither says what this call costs.

Adding a computed risk value to that decision is not new. Quantified risk-adaptive access control appeared in 2007 [11] and was folded into attribute-based models in 2011 [28]. The family is surveyed [4, 10], deployed where a numeric trust value gates access [7], and recently extended to agents, where an LLM judge weighs target-resource risk against its own uncertainty [15]. These models score a subject-object pair: this principal, this resource, this context. Two calls to the same tool by the same agent score alike.

No existing method estimates what a concrete MCP invocation would cost. Most MCP and agent defenses return an enforcement decision, allow or block [1, 27, 46, 51, 59]. Recent systems return richer outcomes — consent levels, verdict classes, calibrated detector scores [20, 31, 52, 55] — but a containment level, an action class and a detection confidence are not magnitudes. Others validate calls at runtime [6], learn access-control policies [1], infer risk from logs [17], or check arguments against a declared intent [41, 58]; none prices the resource the call reaches. Vulnerability scoring does not either: CVSS and AIVSS rate a vulnerability or a system, not a live call [5, 38, 47]. A magnitude matters because it separates calls to allow, calls to review, and calls to refuse [9].

Risk belongs to a (tool, resource) pair, not to the tool: `read_file` is low risk on a public document and high risk on regulated data. A

baseline must combine tool impact, resource sensitivity, and *blast radius* — how far one call reaches. Arguments move that baseline, and the same call reads differently against different histories.

We compute a graded risk score at two times. At design time we score every (tool, resource) pair a server exposes, from tool impact, resource sensitivity, and blast radius. Sensitivity comes from the organization’s own information-classification policy, not from a number assigned without one [36, 48]. At request time we adjust that baseline with the actual arguments, the execution context, and the agent’s recent activity. The score is compared against organizational thresholds, so a call is approved, sent for human review, or refused (§4). Deciding when an agent proceeds alone and when control passes to a person is the central question of adjustable autonomy [40]; the score makes that threshold computable. We implement this as an enforcement framework and evaluate it on tool catalogs captured from real vendor MCP servers, paired with organizational policies, resource inventories, and invocation scenarios (§5).

This paper makes three contributions. First, a **risk-analysis methodology** for assessing the misuse risk of an MCP invocation from tool impact, resource sensitivity, blast radius, runtime arguments, execution context, and the agent’s recent activity, deriving resource sensitivity from organizational policy rather than from a fully labeled asset inventory. Second, an **enforcement framework** that combines the design-time and request-time assessments and uses them to make risk-based access-control decisions for MCP servers. Third, an **evaluation setting** that pairs tool catalogs from real vendor MCP servers with synthesized organizational policies, resource inventories, and invocation scenarios, used to measure whether the calculated risk levels match organizational judgment and whether runtime arguments and behavioral history affect the score as intended.

2 RELATED WORK

MCP Server Security and Risk Assessment. Prior work surveys security risks across the MCP ecosystem [21] and audits open-source MCP servers for software weaknesses, MCP-specific vulnerabilities, and maintainability problems [18]. Several studies examine attacks in which a malicious server, tool description, or external input influences the agent [26, 45, 57]. We study the opposite direction: an agent sends a call that may harm the MCP server or an organizational resource reached through it. Server-to-agent attacks are outside our scope. We assume only that each tool performs the operation represented by its declared interface and resource mapping.

Other work examines the capabilities and weaknesses of MCP servers. Li et al. analyze 2,562 MCP applications and measure their use of file, network, system, and memory APIs [32]. Huang et al. combine static pattern matching with sandboxed execution to detect capabilities such as file access, network requests, and command execution [23]. MCP-in-SoS scans 222 server repositories for code weaknesses, maps them to CWE and CAPEC entries, and derives repository-level risk from the likelihood and severity those catalogs assign [29]. All three assess the server as an artifact, independently of the resources a given installation holds, so the same server receives the same assessment wherever it is deployed.

ConLeash checks whether an MCP call stays within an approved consent boundary [31]. However, it does not estimate the risk or possible impact of a call that is already allowed. Our work addresses this gap by assessing the risk of each invocation from the server side.

Defenses for MCP and tool-using agents. IGAC checks whether an MCP invocation and its arguments remain within the user’s stated intent, while MTGuard compares invocation parameters, runtime behavior, and results against safety policies [58?]. Progent and MiniScope enforce least-privilege authorization, whereas Agent-Guardian learns admissible tool inputs and invocation sequences from normal executions [1, 46, 59]. These systems check whether a tool call follows the user’s request, the security policy, or the agent’s normal behavior. However, a call can pass all these checks and still cause serious damage. For example, an agent may be allowed to delete files, but deleting a temporary file and deleting a critical production file do not have the same risk; our work measures this difference at the MCP server.

Access control. RBAC assigns permissions through roles, while ABAC evaluates attributes of the caller, resource, requested action, and environment conditions [22, 42]. These models can express fine-grained policies, but they do not directly estimate the possible consequence of a permitted request.

Risk-adaptive access control adds a calculated risk value to the authorization decision [11]. Kandala et al. place risk within an attribute-based framework, but leave the concrete risk-calculation method outside their scope [28]. Ni et al. use fuzzy rules over subject and object security information to estimate the risk of an access operation [37]. Shaikh et al. dynamically calculate trust and risk for each subject-object pair from the user’s past behavior [43]. Atlam et al. survey the wider family of risk-based access-control models [4].

For agentic systems, Fleming et al. use an LLM judge to create a task policy, combine static risk values for its tools and resources, and estimate uncertainty [15]. A limitation is that risk is computed for the whole task and is not updated for each call. For example, reading one database record and exporting a full table may use the same tool risk value. Our work instead scores each MCP invocation using its runtime details.

Risk scoring for MCP and agents. CVSS scores the severity of software vulnerabilities. A study of 196 evaluators applying CVSS v3.1 found differences between evaluators, and a follow-up found that 68% assigned a different severity level to vulnerabilities they had scored before [16, 50]. AIVSS extends vulnerability scoring to agentic AI systems [24]. These frameworks score vulnerabilities rather than concrete tool calls.

FIPS 199 and NIST SP 800-60 classify information types and systems by the potential impact of losing confidentiality, integrity, or availability [36, 48]. These classifications provide an organizational view of consequence that our method uses when assessing the resource affected by a call.

Work closer to our setting studies risk at different stages. MCP-in-SoS scores weaknesses in an MCP server repository [29], while AgentTRIM assigns risk levels to tools and applies stronger checks to riskier ones [6]. STARS scores proposed skill invocations using the request, capability, and runtime context before execution [55]. Owiredun-Ashley instead grades the harm of actions recorded after

execution [39]. Together, these methods answer different risk questions: they assess server weaknesses, tool capabilities, invocation safety, or harm after execution. They do not provide a common measure of the consequence to the organization affected by the call. For example, deleting a temporary cache and deleting payroll records may use the same tool and both be irreversible, but their organizational impact is very different. Our work connects each MCP call to the organization’s resource policy and estimates its risk before execution.

3 PROBLEM SETTING AND THREAT MODEL

The setting. An MCP server publishes a catalog of tools over a uniform JSON-RPC channel. A client issues `tools/list` to obtain it — each tool’s name, natural-language description, and JSON argument schema — then invokes one with concrete arguments [3, 35]. The tools are an interface, not the resource: behind them sit the organization’s own repositories, calendars, message histories and payment rails, and the server, not the agent, is the process that touches them. Every invocation is an action the server performs on its own resources at an agent’s request.

Actors. The *server* is the protected party: it absorbs the consequence of anything done to those resources. The *agent* is the risk source, an LLM-driven client choosing which tool to call and with which arguments. The *organization* deploys the server, owns the resources, and publishes the policy that states what losing them would mean. The calling population is open where it counts: a deployment authorises a connection, not an implementation, so the server can inspect neither the model nor the plan behind a request.

How harm arises. Harm arises even though the agent is authenticated and authorised for the tool it calls — through hallucinated calls, risk the agent can articulate but does not act on, capability exceeding what the task needs, injected content, and objectives that diverge from the principal’s [23, 32, 33, 49, 53, 54]. These differ in cause but not on the wire: each produces a well-formed, schema-conforming, authenticated call from a permitted agent to a permitted tool, and no protocol-level check separates them. What varies is what executing it would do. The opposite direction, a malicious server attacking the agent, is out of scope.

What the server observes. At decision time — after a call arrives, before it executes — the server sees the caller’s identity, already consumed by authorization, and the five quantities in Table 1 that describe the invocation itself. Identity is absent from the five because it does not separate the cases: a correct agent, a mistaken one and a manipulated one present the same credentials over the same interface [25, 34]. Nor does the server observe *intent*: it cannot see the user’s instruction, and cannot tell whether an argument naming, say, a \$1,000,000 transfer is a typo, an injection, or a sanctioned operation. The score below is therefore a function of those five alone, never of what the agent meant.

Assumptions. (i) Authentication and transport security are sound and orthogonal: identity is given, and we ask only what that caller may safely do. (ii) The organization publishes a written classification policy of the kind security categorization standards define [36, 48] — named classes defined by adverse impact, plus rules for recognising a resource’s class — and registers the resources its server touches while assigning a sensitivity number to none of them. We

Table 1: What an MCP server can observe about an invocation before executing it. Intent is not among them.

	Observable
t	the tool named in the request
r	the resource(s) that request resolves to
a	the argument values, valid against the tool’s schema
x	session context: the calls already made on this connection
h	the calling agent’s own history of prior invocations

take this to be the realistic case; rules, unlike an inventory, still classify resources created after the policy was written. (iii) A call resolves to rows of that register by its tool and, where several are reachable, by its arguments. (iv) The server can refuse or hold a call, so enforcement is in-band rather than advisory.

The decision problem. Write an invocation as the whole tuple $c = \langle t, r, a, x, h \rangle$ — not the tool alone. Access control supplies a predicate $\text{perm}(c) \in \{0, 1\}$ and already refuses what it rejects, so the population of interest is exactly the calls with $\text{perm}(c) = 1$. The problem is to compute a graded consequence score $\rho(c)$ that orders them by how much damage executing them could do, and to gate on it against thresholds $\theta_r \leq \theta_d$ the deploying organization sets — conditioning authorization on a computed risk value, as risk-adaptive access control does [4, 11, 28]:

$$d(c) = \begin{cases} \text{allow} & \rho(c) < \theta_r \\ \text{review} & \theta_r \leq \rho(c) < \theta_d \\ \text{deny} & \rho(c) \geq \theta_d. \end{cases} \quad (1)$$

With ρ reported as a discrete band the thresholds sit on adjacent bands and review is the band at θ_r ; §5 operates a single flag threshold, swept across bands. Two properties make ρ , not d , the object to design. It must be ordinal rather than binary, so an operator sets where escalation falls instead of inheriting the designer’s cut point [9]; and it must be auditable, decomposed into named factors a security team can argue with, since θ_r and θ_d are an organizational judgement, not a tuned hyperparameter. Section 4 constructs ρ .

4 THE FRAMEWORK

MCPRisk computes the score $\rho(c)$ of §3 in two layers. A static layer builds a baseline for every (tool, resource) pairing a server exposes; a dynamic layer refines it with the concrete arguments, the session, and the calling agent’s own history. The output is an ordinal band.

4.1 Overview

The static layer reads exactly two documents: the tool catalog the server advertises over `tools/list`, and the section of the organization’s policy that covers this server. Neither is a MCPRisk-specific artifact: the server already publishes the catalog, and a security team already maintains the policy. The policy supplies a register of the resources behind the server and the tools that touch them, named classes, and the rules by which the organization’s own staff recognise them; it states no severity number anywhere. The organization’s per-resource sensitivity table never reaches the scorer.

Here it is an evaluation instrument only — the artifact a deployment would not have, which is what lets it serve as an oracle. §5 separates the real captures from what was written for this study.

Three primitives, each on a 1–5 scale, are estimated and multiplied:

$$\text{score} = \text{sensitivity} \times \text{blast} \times \text{impact}, \quad \text{score} \in [1, 125]. \quad (2)$$

The three ask deliberately different questions: impact is about the verb, what one call does; sensitivity about the noun, what the call touches; blast radius about the quantifier, how far the consequence travels. Multiplying rather than averaging keeps a low factor influential: a destructive tool aimed at a worthless resource cannot reach the top of the range, since only a call severe on every axis gets there.

A scan issues four *kinds* of prompt: stage 0 infers the server’s domain once, fixing the vocabulary for what follows; stage 1 assigns tool impact per tool, rules first and the model only on abstention; stage 2 assigns resource sensitivity per register row; stage 3 assigns blast radius per (tool, resource) cell. The last stage dominates a scan’s prompt count — 26 tools over 20 resources is 520 cells. Everything afterwards is deterministic. An assembly pass reconciles twin tools (a bulk variant of a singular tool, or a deprecated alias of a canonical one, is tied to its twin so neither can be priced below the other) and applies a blast floor; a banding function then maps the product onto {low, medium, high, critical}. Banding is not a raw threshold: it encodes explicit security floors — any irreversible operation is at least medium — derived by measuring where an LLM reviewer systematically disagreed with raw thresholds. Determinism buys exactly one thing, and we claim only that: band boundaries, twin reconciliation and the floor do not drift between runs. The residual run-to-run variance sits in the LLM stages, and §5 quantifies it for blast radius, the noisiest of them.

4.2 The Static Layer

The static layer runs once per server, before any agent connects. It scores every (tool, resource) pairing the catalog exposes, producing the baseline that the dynamic layer later refines. Its three primitives are estimated independently and combined by Eq. 2.

4.2.1 Deriving Sensitivity from Policy. The input a risk model most wants is the one an organization is least able to publish. A per-resource sensitivity inventory is expensive to build, sensitive in itself — filenames can be protected health information — and stale the moment a new resource appears. What organizations do publish is a data classification policy.

MCPRisk therefore derives sensitivity by *classify-then-map*, splitting the labour so neither party does the other’s job. The organization states each class’s consequence of loss in its own words — statutory breach, patient injury, privilege waiver, recoverable embarrassment — and writes no numbers. MCPRisk owns the 1–5 scale, maps that adverse-impact language onto it, and classifies each register row into one of the organization’s classes, using the row’s description and the policy’s own recognition and aggregation rules. Classify-then-map follows the security categorization procedure of FIPS 199 and SP 800-60 [36, 48] — impact language first, category second — applied to a document the organization

Table 2: The tool-impact ladder. One question — read, write, or removal — decides the tier.

Tier	Operation
1	none — the server talks about itself: ping, health, version, whoami
2	metadata — names, ids, counts, sizes, timestamps, permissions, schema, listings; consumption state
3	content read or limited write — append a line, add a comment, set one named field
4	ordinary write — the caller supplies what the item says
5	removal or execution — delete, wipe, drop, purge, truncate; execute code; move money

already maintains rather than to a labelled inventory. A rule, unlike an inventory row, still classifies a resource created after the policy was written — a property of the construction, not a result we measure.

4.2.2 Tool Impact: A Deterministic Ladder. Tool impact asks one question of a tool’s own declaration: is this operation a read, a write, or a removal? Table 2 is the resulting ladder, matched by a vocabulary of 281 patterns — 16 for *none*, 52 metadata, 46 read, 122 write, 45 remove — plus 119 ambiguous single words matched against the tool name only, 10 generic read verbs, and 9 parameter detectors. That last scoping matters: matched across a description instead, the ambiguous words make a listing tool look destructive.

The tier 3/4 boundary — limited write against ordinary write — is not invented here: two published standards draw it in the same place. HTTP separates PATCH, a partial modification leaving unmentioned fields untouched [12], from PUT, where the client sends the complete representation [13]; CVSS v4.0 separates integrity impact VI:L, “the amount of modification is limited”, from VI:H, “a total loss of integrity” [14]. Writing a file is a 4; writing one sentence into it is a 3.

Breadth is deliberately absent from the ladder: how many items a call reaches is coverage, which blast radius scores, so the rules read no breadth vocabulary (*all, every, bulk, batch*), no glob and no recursion switch. Parameter detectors are recorded as capability flags but never move a tier: what a caller *could* pass is not what a call *does* pass, which the dynamic layer sees and the static layer cannot. MCP tool annotations are recorded the same way, as evidence and never as a bound [35] — an annotation is a component’s self-assessment rather than an independent measurement, and no component sets the ceiling on its own risk score.

4.2.3 The Rules-to-Model Hand-off. The rules report a confidence with every tier: **0.35** when no tier verb matched, **0.8** when a verb fired, and **0.95** when a verb fired and an MCP annotation corroborated it. The model is consulted only below **0.5**. An annotation therefore raises confidence, and can suppress the model’s review, but never sets or caps the tier itself.

The model is asked where the rules do not know — a write whose declaration never says whether it is bounded — not where two rules disagree, which the precedence order settles. What the arrangement buys is attributability rather than call volume: stages 2 and 3 already spend a prompt per cell, so the tool axis is not where

a scan’s prompt volume lives. Every tier the rules decide traces to a named pattern, and a per-tool field records which scorer decided, so no tier is unattributable. On the catalogs we evaluate the fallback never fired; §5 reports that result and what it stops us claiming.

4.2.4 Blast Radius. Blast radius is scored per (tool, resource) pairing: how far one call’s consequences propagate, which subjects and systems are different afterwards. The rubric numbers that propagation. (1) Almost nobody depends on what was touched; (2) a few subjects; (3) a group, the people or systems attached to what was touched, and the normal case; (4) the whole population inside the resource boundary, the subjects the register says one resource holds; (5) beyond that boundary. Tier 5 requires a route the organization’s own register sanctions: the resource is a hub other systems authenticate against, what the call returns is usable on its own elsewhere, one call reaches the resource’s entire population, or the resource is destroyed with nothing left to restore. Counting items would break on the obvious case: reading one credential file touches one item and compromises every system that credential opens.

One deterministic pass then acts on the number: a floor ties blast radius to the sensitivity of the resource and to irreversible operations. That breaks the primitives’ independence on purpose, in one direction only — sensitivity can raise blast, never lower it — because a call reaching a crown-jewel resource is not a pinpoint consequence whatever its verb. An earlier version also applied a roof; it was removed, because a roof can only ever under-score. Blast radius remains the noisiest of the three stages, which §5 quantifies rather than hides.

4.3 The Dynamic Layer

The static table answers how bad a call would be if it happened; it cannot see the agent, the session, or the arguments actually supplied. The dynamic layer answers how abnormal this call is, from this agent, in this session, given its actual input, using four signals ordered cheap to expensive.

(0) Embedding likelihood. A hashing vectorizer over the tool name, argument tokens and structural flags, reduced with TruncatedSVD fit on the server’s observed history; novelty is the mean distance to the 5 nearest historical calls, calibrated against the history’s own quantiles. **(1) Behavioral baseline.** Deviation from the calling agent’s own history: a tool it has never called, a first touch of a high-sensitivity resource, a burst above its normal rate. **(2) Session sequence risk.** Cumulative sensitivity within a session, plus a flag for a sensitive read followed by an external write in a window — a pattern no per-call view can see. **(3) LLM judge.** Decodes and normalizes arguments (base64, hex, shell expansion) and classifies intent — the only signal that uses a model, and it is optional.

Three of the four are LLM-free, which keeps the layer’s compute cost negligible: fitting takes under 0.5 s per server and scoring roughly 0.05 ms per call. The judge is wired but not exercised here.

Two compositions coexist, entering at different points. The embedding likelihood fills a likelihood slot,

$$\text{final_risk}(c) = \text{score}(c) \times \text{likelihood}(c), \quad \text{likelihood} \in [0.1, 1.0], \quad (3)$$

so that demonstrated-normal behaviour discounts risk rather than erasing it. The 0.1 floor makes that safe: a cell the scan called critical

keeps a tenth of its risk however routine the call looks, and a call with no benign history keeps full static risk at likelihood 1.0, so an agent never observed is never silently discounted. The three band signals s_1, s_2, s_3 — signals (1)–(3) above — then fuse by escalation on top of the banded result, each able only to *raise* it; none can argue a dangerous capability down. Banding first and escalating second gives the quantity §3 asked for:

$$\rho(c) = \text{escalate}(\text{band}(\text{final_risk}(c)), s_1(c), s_2(c), s_3(c)), \quad (4)$$

an ordinal band over the same four labels.

4.4 Enforcement

Enforcement instantiates Eq. 1: $\rho(c)$ is the band just defined and $\theta_r \leq \theta_d$ are the organization’s own, so each is an operating point rather than a redesign. §5 measures the flag/no-flag boundary at both a *high* and a *critical* setting, and reports the benign fall-out each costs. The intermediate review outcome — holding a call for human verification rather than refusing it — belongs to the design and is not evaluated here.

The decision is auditable in the organization’s own terms. A refusal decomposes into the three primitives of Eq. 2, the policy class the resource was mapped to (§4.2.1), the operation tier and the scorer that produced it (§4.2.3), and the likelihood that scaled the result. Two of the three primitives are model-assigned, so this is not a model-free audit trail; what the decomposition gives an operator is a named class and a named operation to dispute, in place of a single opaque number.

5 EVALUATION

Four questions organise the evaluation. **RQ1:** can an organization’s own severity judgements be recovered from its policy prose alone? **RQ2:** do the deterministic impact rules suffice, or does tool impact need a model? **RQ3:** does the request-time signal add discrimination over the design-time baseline? **RQ4:** does the calibration survive contact with real traffic?

Three corpora, one per question group. RQ1 and RQ2 run on three vendor MCP servers: a calendar server, GitHub, and Slack. Their tool catalogs are genuine, captured from the live vendor servers through the protocol’s own tools/list call [35], and are used exactly as returned — 13 tools for calendar, 26 for GitHub, 16 for Slack, 55 in total. RQ3 runs on a different corpus: a session testbed spanning ten server profiles, whose catalogs and whose sessions alike were written for this study, so RQ3 sits entirely on the synthesized side of the accounting below and shares no catalog with RQ1 or RQ2. RQ4 runs on a further server, a live `mcp-server-git` driven over stdio, and is the one genuinely real execution trace in the section.

What is synthesized. Everything the captured catalogs do not supply. The organizational policy document, the registers that name the resources behind each of the three vendor servers (16, 20 and 20 resources, 56 in total), and the ten RQ3 server profiles together with the invocation scenarios over them were all written for this study. This is therefore an *evaluation setting*, not a benchmark, and we do not present it as one. It exists because graded per-action severity ground truth for MCP invocations barely exists: we know

Table 3: RQ1 — derived resource sensitivity against the organization’s held-out per-resource table. The derivation reads the policy document only.

Server	Resources	MAE	Exact	Within 1
calendar	16	0.125	88%	100%
github	20	0.10	90%	100%
slack	20	0.10	90%	100%

of no external benchmark that supplies it, which is also why RQ3 has no external comparison point.

Protocol for RQ1 and RQ2. The scorer is given exactly two documents: the captured tool catalog and the relevant section of the organizational policy. The per-resource sensitivity table that ships with each server profile is *held out* and never enters the pipeline; it is the ground truth. That table is the organization’s own 1–5 rating of each register row, and the policy document from which we derive our rating states no number anywhere. We report mean absolute error, exact agreement, and agreement to within one tier. The within-one column is reported because the sensitivity rating is not consumed on its own: it is one of three 1–5 primitives whose product spans 1–125 before banding (§4), so we evaluate each primitive against the organization’s judgement at the resolution the primitive is actually authored at.

RQ1: policy text recovers the organization’s severities. Table 3 gives the result per server. Across all 56 resources, 50 match the organization’s own rating exactly, and *none is off by more than one tier* on any of the three servers. Mean absolute error is 0.125 on calendar and 0.10 on both GitHub and Slack. The uniform 100% within-one column is the load-bearing number: a derivation that reads only prose does not merely correlate with the withheld table, it lands on it or immediately beside it, everywhere. Exact agreement of 88–90% is the weaker claim and the more honest headline, because six resources do move.

Where it drifts. Table 4 lists all six misses. Every one is adjacent-tier and every one has a readable cause, which is the useful part. Two are register-layout effects: `account-directory` inherits severity from the Restricted-class account configuration it sits beside, and `repository-catalog` is rated up because a full catalog enumeration maps the estate. Two are classification-language effects: `infra-config` is read as serious lasting harm rather than as control plane, and `calendar-records` is read as a bare attribute listing. Two follow the policy’s own qualifiers: `incident-response` carries a self-sufficient flag and credentials pasted mid-incident, and `research-team` holds unpublished research read as competitively damaging. The misses run in both directions, the majority above the organization’s own rating rather than below — the direction a security control should err in — though six cases establish a reading, not a calibrated bias.

RQ2: the impact ladder needed no model. Tool impact is the one stage where a model is optional by construction — the rules answer, and the model is consulted only when the rules abstain (§4). Table 5 reports what happened. The deterministic ladder decided all 55 tools across the three servers and the LLM fallback was invoked zero times. Compared against an earlier arm

Table 4: RQ1 — all six disagreements. Every miss is one tier, and every miss is traceable to a specific reading of the policy text.

Resource	Srv.	Ours	Org	Reading
<code>account-directory</code>	cal	5	4	sits beside the Restricted-class account configuration
<code>calendar-records</code>	cal	2	3	read as a bare attribute listing
<code>infra-config</code>	gh	4	5	serious lasting harm, not control plane
<code>repository-catalog</code>	gh	3	2	full enumeration maps the estate
<code>incident-response</code>	sl	5	4	self-sufficient flag, credentials mid-incident
<code>research-team</code>	sl	4	3	unpublished research, competitively damaging

Table 5: RQ2 — tool impact. The rules answered every tool; the LLM fallback never fired. Agreement is against an earlier arm in which the LLM scored tool impact.

Server	Tools	Ladder	LLM	Agrees
calendar	13	13	0	13/13
github	26	26	0	24/26
slack	16	16	0	14/16

in which tool impact was scored by the LLM, the ladder agrees on 51 of 55 tools (93%), and all four disagreements are one tier: `get_pull_request_files` (3 vs. 2), `search_users` (3 vs. 2), `conversations_leave` (4 vs. 5) and `usergroups_me` (5 vs. 4). The consequence is that the tool axis can be decided without a model call, at 93% agreement with the LLM arm it replaces and with every disagreement confined to one tier.

An honesty constraint on RQ2. The fallback did not fire, and the reason matters more than the result. Every one of the 55 tools matched a tier verb, so every rule confidence was 0.8. The 0.95 value was unreachable, because none of the three vendor catalogs declares MCP tool annotations; and no tool fell to the 0.35 abstention value that would have handed it to the model (§4.2.3). On this corpus the rules-to-model hand-off is therefore covered by unit tests only. We claim that the ladder was sufficient here; we do not claim the hybrid was exercised.

RQ3: the request-time signal adds discrimination. The scenario generator partitions sessions three ways. Adversarial sessions are the target class; benign sessions are the population that must not be disturbed; and *misuse* sessions are accidental-insider behaviour, constructed to sit between the two, which gives the measurement an interior class rather than only a separable pair. The ten server profiles contribute 86–498 benign and 4–20 adversarial sessions each, 88 adversarial sessions in total. Table 6 reports the two operating thresholds an organization would realistically choose, high and critical. Each session is scored twice — static only, and static combined with the request-time signal — so the difference is exactly what the dynamic layer contributes; a session takes the band of its riskiest call.

Table 6: RQ3 — session-level discrimination, static versus static combined with the request-time signal, over ten synthesized server profiles (86–498 benign and 4–20 adversarial sessions each).

Scorer	Recall (adv.)	Flagged (misuse)	Fall-out (benign)
<i>Flag threshold = high</i>			
static	91%	63%	49%
combined	100%	71%	49%
<i>Flag threshold = critical</i>			
static	56%	24%	9%
combined	81%	24%	9%

The critical row is the result. Holding benign fall-out fixed at 9%, adversarial recall rises from 56% to 81%, and 22 of the 88 adversarial sessions are caught *only* by the dynamic signal. That is 25 points of recall bought at no cost in false alarms, which is the shape an operator needs: the static scorer alone reaches comparable recall only by dropping to the permissive threshold, where it recovers 91% but flags 49% of benign sessions. At that permissive threshold the dynamic layer still helps and is merely saturated, 91% to 100% recall with 8 of 88 caught only dynamically.

The interior class does not move at the strict threshold. We built misuse in as the hard case, so its row is the one to read honestly. At the critical threshold the dynamic layer buys nothing on it — 24% flagged by the static scorer, 24% by the combined one — in the same sweep where adversarial recall gains 25 points. Misuse improves only at the permissive threshold, 63% to 71%. On this testbed the request-time signal separates deliberate adversarial behaviour and leaves accidental-insider behaviour where the static scorer left it. We did not run the ablation that would attribute this to a particular signal, so we report it as a bound on the claim rather than explain it away: the layer buys adversarial recall, not insider recall.

Mean session band-rank (low = 1 to critical = 4) moves from static to combined as follows: adversarial 3.47 to 3.81, misuse 2.84 to 2.92, benign 2.41 to 2.41. Benign sessions are unchanged on both measures, band-rank and fall-out, which is the property an operator needs before turning the layer on. Two design constraints bound how far the layer can move anything at all: the band signals can only raise risk above the static floor, and the likelihood multiplier is bounded below at 0.1, so no amount of apparent normality discounts a critical cell to nothing.

RQ4: the calibration transfers to real traffic. Everything in RQ3 is synthetic, so we repeated the calibration check on genuinely real MCP traffic. We drove a live mcp-server-git over studio against the implementation’s own repository (223 real commits) and captured 400 real tool calls, of which 399 executed without error, exercising 12 git tools and 106 distinct call signatures. Arguments come from the repository’s actual history: real 40-char SHAs, real paths, real branch names. All 400 captured calls are benign by construction, being script-driven replay against that history; the only non-benign items in RQ4 are three calls we injected. We fitted on the first 70% of the calls chronologically and scored the held-out 30%. The false-positive rate on benign traffic was 0.8%,

Table 7: Blast radius is the noisy stage: (tool, resource) cells that changed between arms under the same rubric and the same resource identifiers.

Server	Cells	Diff.	Raised	Lower.	→N/A	N/A→
calendar	208	40	11	3	15	11
github	520	100	26	21	32	21
slack	320	72	30	18	16	8

against a design target of 1–2%, and the mean held-out likelihood was 0.107 — essentially everything sits on the multiplier’s 0.1 floor, which is the intended behaviour for traffic that looks like its own history. The three injected calls were a 200-char commit message, a bare git_reset, and a fabricated exfiltrate_all carrying a payload; all three scored high, against held-out benign traffic sitting on that floor. The q99 quantile-anchored ramp was tuned on the synthetic calendar, GitHub and Slack corpus and transferred to real git traffic with no adjustment.

Limitations of the measurement. Blast radius is the noisy stage. Table 7 compares the (tool, resource) cells produced with and without the policy context: 19–22% of cells move, with roughly half the movement being the relevance gate flipping a cell into or out of not-applicable. An earlier arm measured 23–35 cells per server of pure run-to-run variance with the prompt held fixed. Calendar’s 40 changed cells sit just above that band, but GitHub’s 100 and Slack’s 72 are well above it, so on the two larger servers the movement cannot be attributed to run-to-run variance alone; we nonetheless do not read individual cell changes as signal. The movement is not a uniform discount either: summed scores rise on calendar (2280 to 2697) and Slack (3779 to 4096) and fall on GitHub (6340 to 5328). Three further limits bound the claims. The RQ3 labels are synthetic by construction — the benign, misuse and adversarial partition is a property of the generator, not an independent annotation — so RQ3 measures separation under a known generative model, and RQ4 only checks that the calibration holds elsewhere. The optional LLM judge in the request-time layer was wired but not run in these experiments. And the static result rests on one policy document over three servers, so cross-organization generalisation is untested; beyond catalog capture, the vendor servers themselves were not driven live.

REFERENCES

- [1] Nadya Abaev, Denis Klimov, Gerard Levinov, David Mimran, Yuval Elovici, and Asaf Shabtai. 2026. AgentGuardian: Learning Access Control Policies to Govern AI Agent Behavior. *arXiv preprint arXiv:2601.10440* (2026).
- [2] Shames Al Mandalawi, Muzakkiruddin Ahmed Mohammed, Hendrika Maclean, Mert Can Cakmak, and John R. Talburt. 2025. Policy-Aware Generative AI for Safe, Auditable Data Access Governance. In *17th International Conference on Knowledge and Systems Engineering (KSE)*. arXiv:2510.23474.
- [3] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Accessed 2026-07-30.
- [4] Hany F. Atlam, Muhammad Ajmal Azad, Madini O. Alassafi, Abdulrahman A. Alshdadi, and Ahmed Alenezi. 2020. Risk-Based Access Control Model: A Systematic Literature Review. *International Journal of Computer Network and Information Security* 12, 5 (2020).
- [5] Atmane Ayoub Mansour Bahar and Ahmad Samer Wazan. 2024. On the Validity of Traditional Vulnerability Scoring Systems for Adversarial Attacks against LLMs. *arXiv preprint arXiv:2412.20087* (2024).
- [6] Roy Betser, Shamik Bose, Amit Giloni, Chiara Picardi, Sindhu Padakandla, and Roman Vainshtein. 2026. AgentTRIM: Tool Risk Mitigation for Agentic AI. *arXiv preprint arXiv:2601.12449* (2026).

- [7] Leonard Bradatsch, Aleksandr Miroshkin, Natasa Trkulja, and Frank Kargl. 2024. Zero Trust Score-Based Network-Level Access Control in Enterprise Networks. *arXiv preprint arXiv:2402.08299* (2024).
- [8] Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. 2026. AgentBound: Securing Execution Boundaries of AI Agents. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), Article FSE096. <https://doi.org/10.1145/3808103> arXiv:2510.21236.
- [9] Tri Cao, Minh-Huy Trinh, Ailin Deng, Quoc-Nam Nguyen, Khoa Duong, Ngai-Man Cheung, and Bryan Hooi. 2024. Are Anomaly Scores Telling the Whole Story? A Benchmark for Multilevel Anomaly Detection. *arXiv preprint arXiv:2411.14515* (2024).
- [10] Pavlos Cheimonidis and Konstantinos Rantos. 2023. Dynamic Risk Assessment in Cybersecurity: A Systematic Literature Review. *Future Internet* 15, 10 (2023). <https://doi.org/10.3390/fi15100324>
- [11] Pau-Chen Cheng, Pankaj Rohatgi, Claudia Keser, Paul A. Karger, Grant M. Wagner, and Angela Schuett Reninger. 2007. Fuzzy Multi-Level Security: An Experiment on Quantified Risk-Adaptive Access Control. In *IEEE Symposium on Security and Privacy (S&P)*. <https://doi.org/10.1109/SP.2007.21>
- [12] Lisa Dusseault and James M. Snell. 2010. *PATCH Method for HTTP*. Technical Report RFC 5789. Internet Engineering Task Force. <https://doi.org/10.17487/RFC5789>
- [13] Roy T. Fielding, Mark Nottingham, and Julian Reschke. 2022. *HTTP Semantics*. Technical Report RFC 9110. Internet Engineering Task Force. <https://doi.org/10.17487/RFC9110>
- [14] FIRST.org. 2023. Common Vulnerability Scoring System Version 4.0: Specification Document. <https://www.first.org/cvss/v4-0/specification-document>. Accessed 2026-07-31.
- [15] Charles Fleming, Ashish Kundu, and Ramana Kompella. 2025. Uncertainty-Aware, Risk-Adaptive Access Control for Agentic Systems using an LLM-Judged TBAC Model. *arXiv preprint arXiv:2510.11414* (2025).
- [16] Forum of Incident Response and Security Teams (FIRST). 2023. *Common Vulnerability Scoring System Version 4.0: Specification Document*. Technical Report. FIRST. <https://www.first.org/cvss/v4.0/specification-document> Document Version 1.2.
- [17] Jiayi Fu, Yuansen Zhang, and Yinggui Wang. 2025. Can LLM Infer Risk Information From MCP Server System Logs? *arXiv preprint arXiv:2511.05867* (2025).
- [18] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Bram Adams, and Ahmed E. Hassan. 2025. Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers. *arXiv preprint arXiv:2506.13538* (2025).
- [19] Ping He, Yuexiang Xie, Yaliang Li, and Shouling Ji. 2026. Hybrid Analysis for Secure MCP Tool Use in LLM Agents. *arXiv preprint arXiv:2607.25297* (2026).
- [20] Elias Hossain, Md Mehedi Hasan Nipu, Tasfia Nuzhat Ornee, Rajib Rana, and Niloofer Yousefi. 2026. NEXUS: Structured Runtime Safety for Tool-Using LLM Agents. *arXiv preprint arXiv:2607.19356* (2026).
- [21] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2026. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Transactions on Software Engineering and Methodology* (2026). <https://doi.org/10.1145/3796519> arXiv:2503.23278.
- [22] Vincent C. Hu, David Ferraio, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Technical Report NIST Special Publication 800-162. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-162>
- [23] Charoes Huang, Xin Huang, and Amin Milani Fard. 2026. Auditing MCP Servers for Over-Privileged Tool Capabilities. *arXiv preprint arXiv:2603.21641* (2026).
- [24] Ken Huang, Michael Bargury, Vineeth Sai Narajala, Bhavya Gupta, and Tim Marple. 2026. *AIVSS Scoring System for OWASP Agentic AI Core Security Risks*. Technical Report Version 0.8. OWASP Foundation.
- [25] Yuhang Huang, Boyang Ma, Biwei Yan, Xuelong Dai, Yechao Zhang, Minghui Xu, Kaidi Xu, and Yue Zhang. 2026. Give Them an Inch and They Will Take a Mile: Understanding and Measuring Caller Identity Confusion in MCP-Based AI Systems. *arXiv preprint arXiv:2603.07473* (2026).
- [26] Invariant Labs. 2025. MCP Security Notification: Tool Poisoning Attacks. <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>. Accessed 2026-07-30.
- [27] Huihao Jing, Haoran Li, Wenbin Hu, Qi Hu, Xu Heli, Tianshu Chu, Peizhao Hu, and Yangqiu Song. 2025. MCPi: Protecting MCP Safety via Model Contextual Integrity Protocol. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://aclanthology.org/2025.emnlp-main.62/>
- [28] Savith Kandala, Ravi Sandhu, and Venkata Bhamidipati. 2011. An Attribute Based Framework for Risk-Adaptive Access Control Models. In *Sixth International Conference on Availability, Reliability and Security (ARES)*. <https://doi.org/10.1109/ARES.2011.41>
- [29] Pratyay Kumar, Miguel Antonio Guirao Aguilera, Srikanthayani Srikanthswara, Satyajayant Misra, and Abu Saleh Md Tayeen. 2026. MCP-in-SoS: Risk Assessment Framework for Open-Source MCP Servers. *arXiv preprint arXiv:2603.10194* (2026).
- [30] Xinfeng Li, Dong Huang, Jie Li, Hongyi Cai, Zhenhong Zhou, Wei Dong, Xiaofeng Wang, and Yang Liu. 2025. A Vision for Access Control in LLM-based Agent Systems. *arXiv preprint arXiv:2510.11108* (2025).
- [31] Ying Li, Yanju Chen, Peiran Wang, Issac Khabra, Faysal Hossain Shezan, Yu Feng, and Yuan Tian. 2026. Options, Not Clicks: Lattice Refinement for Consent-Driven MCP Authorization. *arXiv preprint arXiv:2605.11360* (2026).
- [32] Zhihao Li, Kun Li, Boyang Ma, Minghui Xu, Yue Zhang, and Xiuzhen Cheng. 2025. We Urgently Need Privilege Management in MCP: A Measurement of API Usage in MCP Ecosystems. In *IEEE International Conference on Mobile Ad-Hoc and Smart Systems (MASS)*. arXiv:2507.06250.
- [33] Aengus Lynch, Benjamin Wright, Caleb Larson, Stuart J. Ritchie, Soren Minder-mann, Evan Hubinger, et al. 2025. Agentic Misalignment: How LLMs Could Be Insider Threats. *arXiv preprint arXiv:2510.05179* (2025).
- [34] David Mellafe Zuvic. 2026. Capability Gates Are Not Authorization: Confused-Deputy Failures in LLM Agent Frameworks. *arXiv preprint arXiv:2606.28679* (2026).
- [35] Model Context Protocol. 2025. Model Context Protocol Specification. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>. Revision 2025-06-18. Accessed 2026-08-02.
- [36] National Institute of Standards and Technology. 2004. *Standards for Security Categorization of Federal Information and Information Systems*. Technical Report FIPS PUB 199. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.199>
- [37] Qun Ni, Elisa Bertino, and Jorge Lobo. 2010. Risk-Based Access Control Systems Built on Fuzzy Inferences. In *Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*. 250–260. <https://doi.org/10.1145/1755688.1755719>
- [38] OWASP Foundation. 2025. OWASP AI Vulnerability Scoring System (AIVSS). <https://aivss.owasp.org/>. Accessed 2026-07-30.
- [39] Harry Owiredu-Ashley. 2026. Beyond Attack-Success Rate: Action-Graded Severity Scale for Tool-Using AI Agents. *arXiv preprint arXiv:2607.07474* (2026).
- [40] David V. Pynadath, Paul Scerri, and Milind Tambe. 2002. Towards Adjustable Autonomy for the Real World. *Journal of Artificial Intelligence Research* 17 (2002), 171–228. <https://doi.org/10.1613/jair.1037>
- [41] Md Ashikur Rahman, Md Arifur Rahman, Niamul Hassan Samin, Khandaker Rifah Tasnia, Sifat Rahman Ahona, and Juena Ahmed Noshin. 2026. Beyond Aggregate Risk: Role-Stratified Conformal Risk Control for LLM Tool Calls. *arXiv preprint arXiv:2607.24343* (2026).
- [42] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *Computer* 29, 2 (1996), 38–47. <https://doi.org/10.1109/2.485845>
- [43] Riaz Ahmed Shaikh, Kamel Adi, and Luigi Logrippo. 2012. Dynamic Risk-Based Decision Methods for Access Control Systems. *Computers & Security* 31, 4 (2012), 447–464. <https://doi.org/10.1016/j.cose.2012.02.006>
- [44] Guanquan Shi, Haohua Du, Zhiqiang Wang, Xiaoyu Liang, Weiwenpei Liu, Song Bian, and Zhenyu Guan. 2025. SoK: Trust-Authorization Mismatch in LLM Agent Interactions. *arXiv preprint arXiv:2512.06914* (2025). Revised February 2026.
- [45] Jiawen Shi, Zenghui Yuan, Guiyao Tie, et al. 2025. Prompt Injection Attack to Tool Selection in LLM Agents. *arXiv preprint arXiv:2504.19793* (2025).
- [46] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025).
- [47] Jonathan M. Spring, Eric Hatleback, Allen D. Householder, Art Manion, and Deana Shick. 2021. Time to Change the CVSS? *IEEE Security & Privacy* (2021). <https://doi.org/10.1109/MSEC.2020.3044475>
- [48] Kevin Stine, Rich Kissel, William C. Barker, Jim Fahlsing, and Jessica Gulick. 2008. *Guide for Mapping Types of Information and Information Systems to Security Categories*. Technical Report NIST Special Publication 800-60 Vol. 1, Rev. 1. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-60v1r1>
- [49] Yuzhi Tang, Tianxiao Li, Elizabeth Li, Chris J. Maddison, Honghua Dong, and Yangjun Ruan. 2025. LM Agents May Fail to Act on Their Own Risk Knowledge. *arXiv preprint arXiv:2508.13465* (2025).
- [50] Julia Wunder, Andreas Kurtz, Christian Eichenmüller, Freya Gassmann, and Zinaida Benenson. 2024. Shedding Light on CVSS Scoring Inconsistencies: A User-Centric Study on Evaluating Widespread Security Vulnerabilities. In *2024 IEEE Symposium on Security and Privacy (SP)*. 1102–1121. <https://doi.org/10.1109/SP54263.2024.00058>
- [51] Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. 2025. MCP-Guard: A Multi-Stage Defense-in-Depth Framework for Securing Model Context Protocol in Agentic AI. *arXiv preprint arXiv:2508.10991* (2025).
- [52] Chenglin Yang. 2026. AgentTrust: Runtime Safety Evaluation and Interception for AI Agent Tool Use. *arXiv preprint arXiv:2605.04785* (2026).

- [53] Chenlong Yin, Zeyang Sha, Shiwen Cui, Changhua Meng, and Zechao Li. 2025. The Reasoning Trap: How Enhancing LLM Reasoning Amplifies Tool Hallucination. *arXiv preprint arXiv:2510.22977* (2025).
- [54] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. arXiv:2403.02691.
- [55] Guijia Zhang, Shu Yang, Xilin Gong, and Di Wang. 2026. STARS: Skill-Triggered Audit for Request-Conditioned Invocation Safety in Agent Systems. *arXiv preprint arXiv:2604.10286* (2026).
- [56] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2026. Parasites in the Toolchain: A Large-Scale Analysis of Attacks on the MCP Ecosystem. In *IEEE Symposium on Security and Privacy (S&P)*. arXiv:2509.06572.
- [57] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2026. Parasites in the Toolchain: A Large-Scale Analysis of Attacks on the MCP Ecosystem. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE. <https://doi.org/10.1109/SP63933.2026.00154>
- [58] Genliang Zhu and Chu Wang. 2026. Intent-Governed Tool Authorization for AI Agents. *arXiv preprint arXiv:2606.22916* (2026).
- [59] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, et al. 2025. MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents. *arXiv preprint arXiv:2512.11147* (2025).